Pseudocode

Basic Iteration

Pseudocode is a simple, language-independent way of writing an algorithm. It uses structured words such as Start, while, if, and End to describe a computation clearly, without following the strict rules of a real programming language.

1. Why use pseudocode?
Flowcharts are useful because they show the flow of a process visually. However, for larger problems they can become too large, difficult to edit, and hard to share or update in a team.
Pseudocode solves this by expressing the same logic as readable . It is easier to modify, compare, and convert into actual code later.

2. Basic iteration structure
Basic iteration means repeatedly performing a set of steps until a condition becomes false.
The general structure is:

```
Start
    Initialise variables
    while (condition is true) {
        Do the repeated work
    }
    Do any final work
End
```

The while statement checks a condition before each repetition:

- If the condition is true, the block inside { } runs.
- If it is false, the loop stops.
- The block of statements between { and } is called the **block of code**.

3. Example: Counting cards
Suppose we have a pile of cards and want to count how many cards there are.

```
Start
    count = 0
    while (Pile1 has more cards) {
        Pick a card X from Pile1
        Move X to Pile2
        count = count + 1
    }
End
```

Explanation

Step	Meaning
count = 0	Initially, no cards have been counted
while (Pile1 has more cards)	Continue while unseen cards remain
Pick a card X	Take one card from the unseen pile
Move X to Pile2	Mark it as seen
count = count + 1	Increase the count by 1

The statement count = count + 1 does **not** mean that count equals itself plus one. It means: take the current value of count, add 1, and store the result back in count.

4. Example: Finding the sum
To find the sum of a field F from every card, replace the counting update with an addition.

```
Start
    sum = 0
    while (Pile1 has more cards) {
        Pick a card X from Pile1
        Move X to Pile2
        sum = sum + X.F
    }
End
```

Here, X.F means the value of field F in card X.
For example:

- To find total Maths marks, use X.mathsMarks.
- To find total bill amount, use X.billAmount.
- To find total letters, use X.letterCount.

5. Example: Finding the average
An average needs both a total and a count.

```
Start
    count = 0
    sum = 0
    while (Pile1 has more cards) {
        Pick a card X from Pile1
        Move X to Pile2
        count = count + 1
        sum = sum + X.F
    }
    average = sum / count
End
```

Inside the loop, we update only count and sum. After the loop ends, we calculate: average=sumcountaverage=countsum
This is efficient because all cards are processed in one pass.

6. Important points

- Start marks the beginning of the algorithm.
- End marks the end.
- = is an **assignment** operator; it stores a value in a variable.
- while repeats a block until its condition becomes false.
- { } groups the statements that must repeat.
- Variables such as count, sum, and average store intermediate results.
- The loop must eventually stop; otherwise, it becomes an infinite loop.

Quick comparison

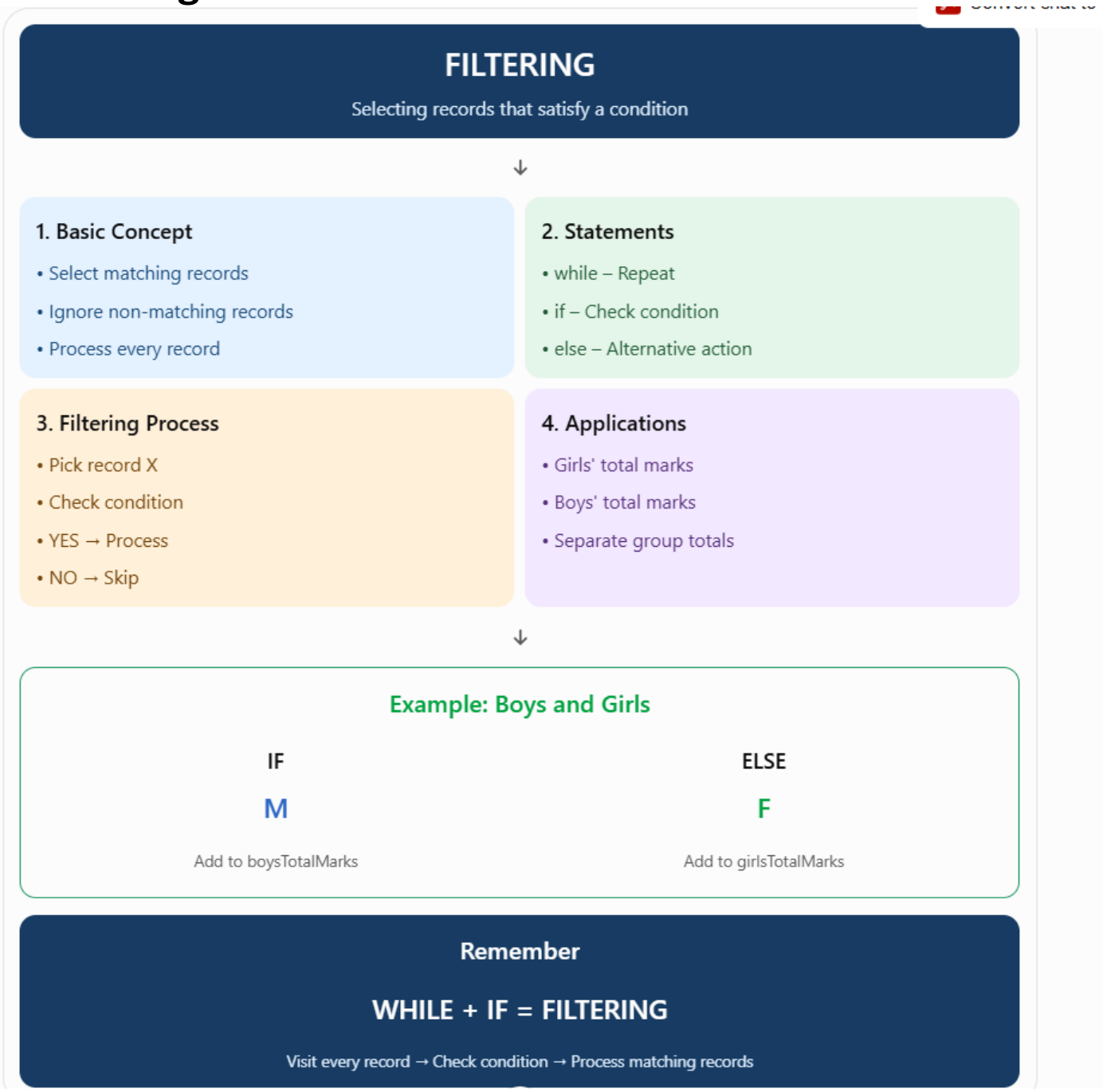
Task	**Initial value**	**Repeated update**

Task	**Initial value**	**Repeated update**
Count cards	count = 0	count = count + 1
Find sum	sum = 0	sum = sum + X.F

Task	Initial value	Repeated update
Find average	count = 0, sum = 0	Update both, then divide after the loop
Find maximum	max = 0	if X.F > max: max = X.F

Basic iteration is the foundation for filtering, compound conditions, and dynamic filtering in later pseudocode.

Filtering



Filtering means selecting only the data items that satisfy a particular condition and ignoring the rest. In pseudocode, this is done using an if statement inside a while loop.

1. Basic idea

We process every card or record, but perform an action only when the record passes the filter condition.

```
For each record X:
    If (condition is true) {
        Do the required action
    }
```

If the condition is false, the record is skipped.

2. Filtering with if

An if statement executes a block once when its condition is true. This is different from while, which repeats a block until its condition becomes false.

```
if (condition) {
    Do something
}
```

An optional else block can specify what to do when the condition is false:

```
if (condition) {
    Do this
} else {
    Do that
}
```

3. Example: Total marks of girls

Suppose we want the total marks of all girls in a classroom dataset.

```
Start
    girlsTotalMarks = 0
    while (Pile1 has more cards) {
        Pick a card X from Pile1
        Move X to Pile2
        if (X.gender == "F") {
            girlsTotalMarks = girlsTotalMarks + X.totalMarks
        }
    }
End
```

Here, the filter condition is:

```
X.gender == "F"
```

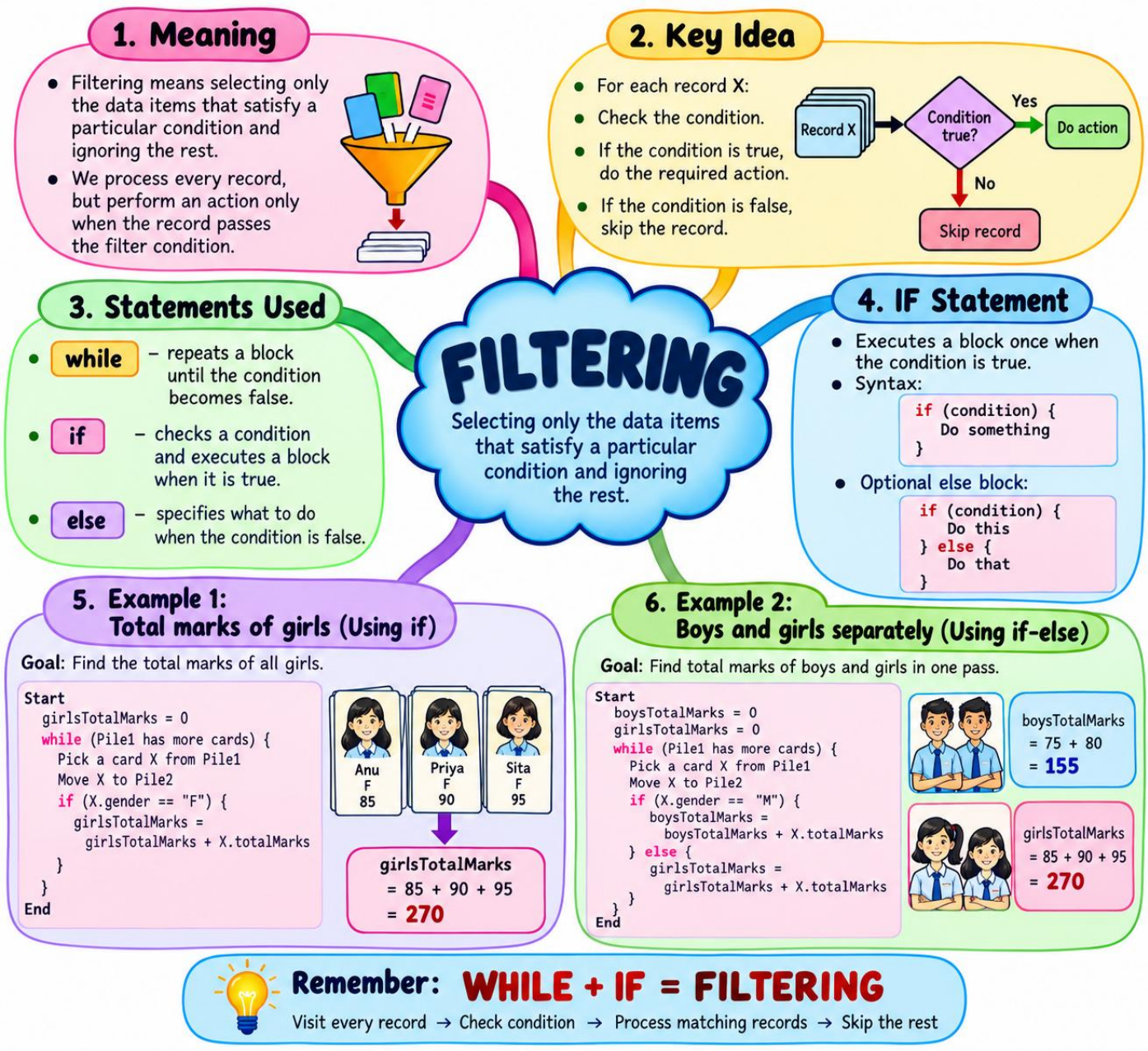
Only cards belonging to girls are added to girlsTotalMarks; all other cards are ignored.

4. Example: Boys and girls separately

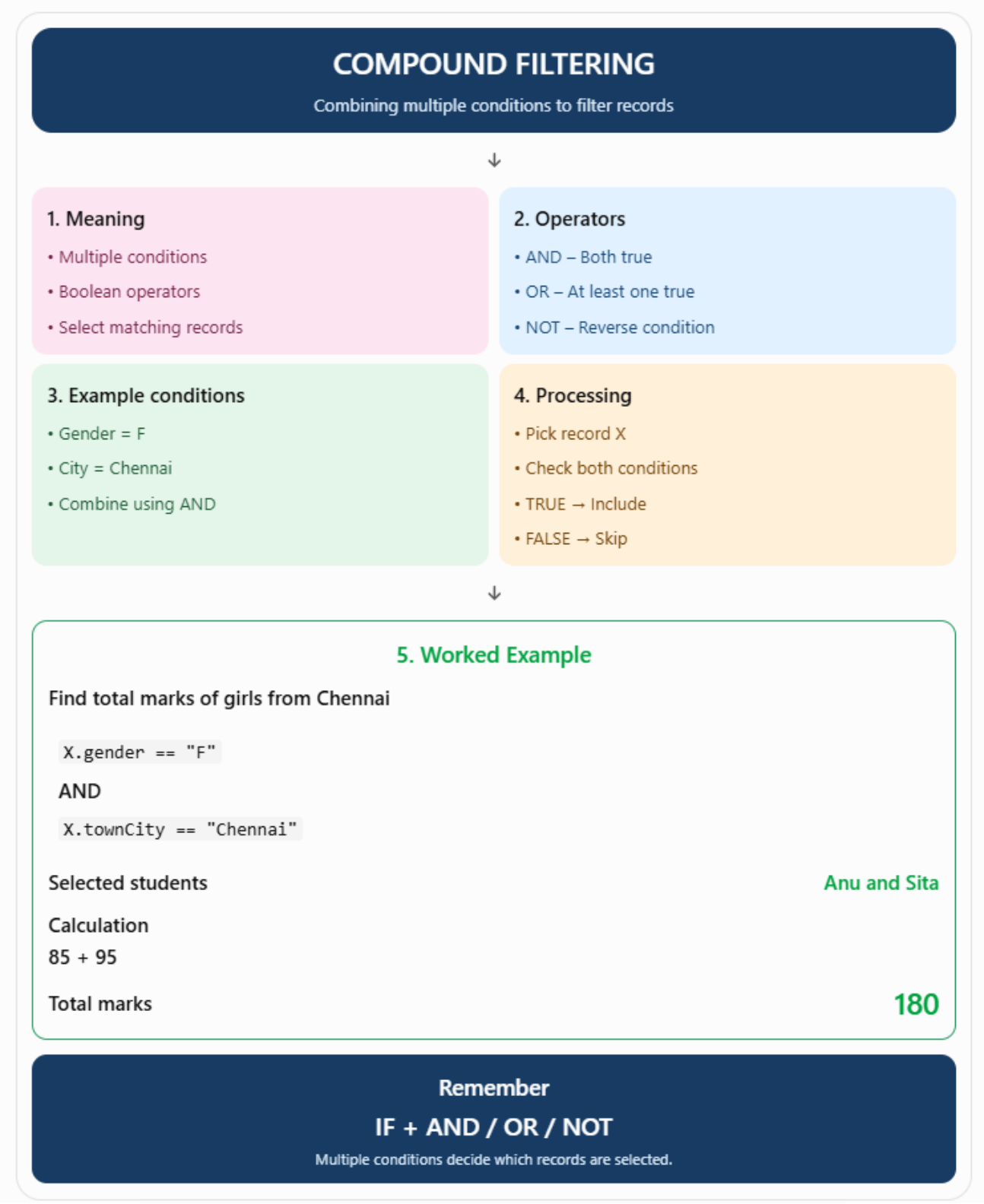
We can use if and else to process both groups in one pass.

```
Start
    boysTotalMarks = 0
    girlsTotalMarks = 0
    while (Pile1 has more cards) {
        Pick a card X from Pile1
        Move X to Pile2
        if (X.gender == "M") {
            boysTotalMarks = boysTotalMarks + X.totalMarks
        } else {
            girlsTotalMarks = girlsTotalMarks + X.totalMarks
        }
    }
End
```

- If the gender is M, the marks are added to boysTotalMarks.
- Otherwise, the marks are added to girlsTotalMarks.



5. Compound filtering conditions



Sometimes a record must satisfy more than one condition. We combine conditions using Boolean operators:

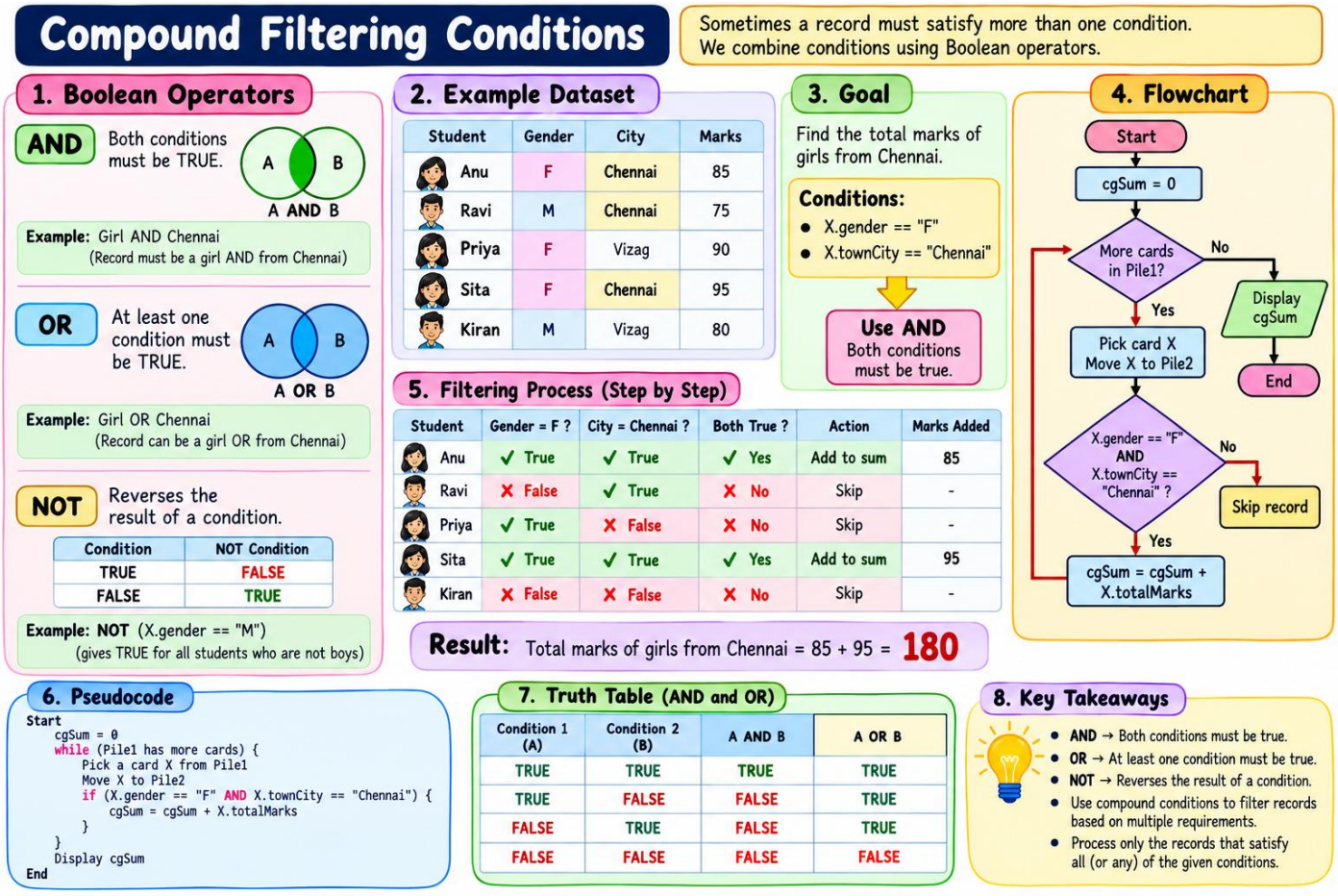
- AND: both conditions must be true.
- OR: at least one condition must be true.
- NOT: reverses a true condition to false, and vice versa.

Example: Girls from Chennai

To find the total marks of girls from Chennai, both conditions must be true:

```
X.gender == "F"
X.townCity == "Chennai"
Start
    cgSum = 0
    while (Pile1 has more cards) {
        Pick a card X from Pile1
        Move X to Pile2
        if (X.gender == "F" AND X.townCity == "Chennai") {
            cgSum = cgSum + X.totalMarks
        }
    }
End
```

The condition uses AND, so a card is processed only if the student is both a girl and from Chennai.



6. Filtering without else

If we want to process only one selected group, we do not need else.

For example, to find the total marks of girls only:

```
if (X.gender == "F") {
    girlsTotalMarks = girlsTotalMarks + X.totalMarks
}
```

If the condition is false, the loop simply moves to the next card.

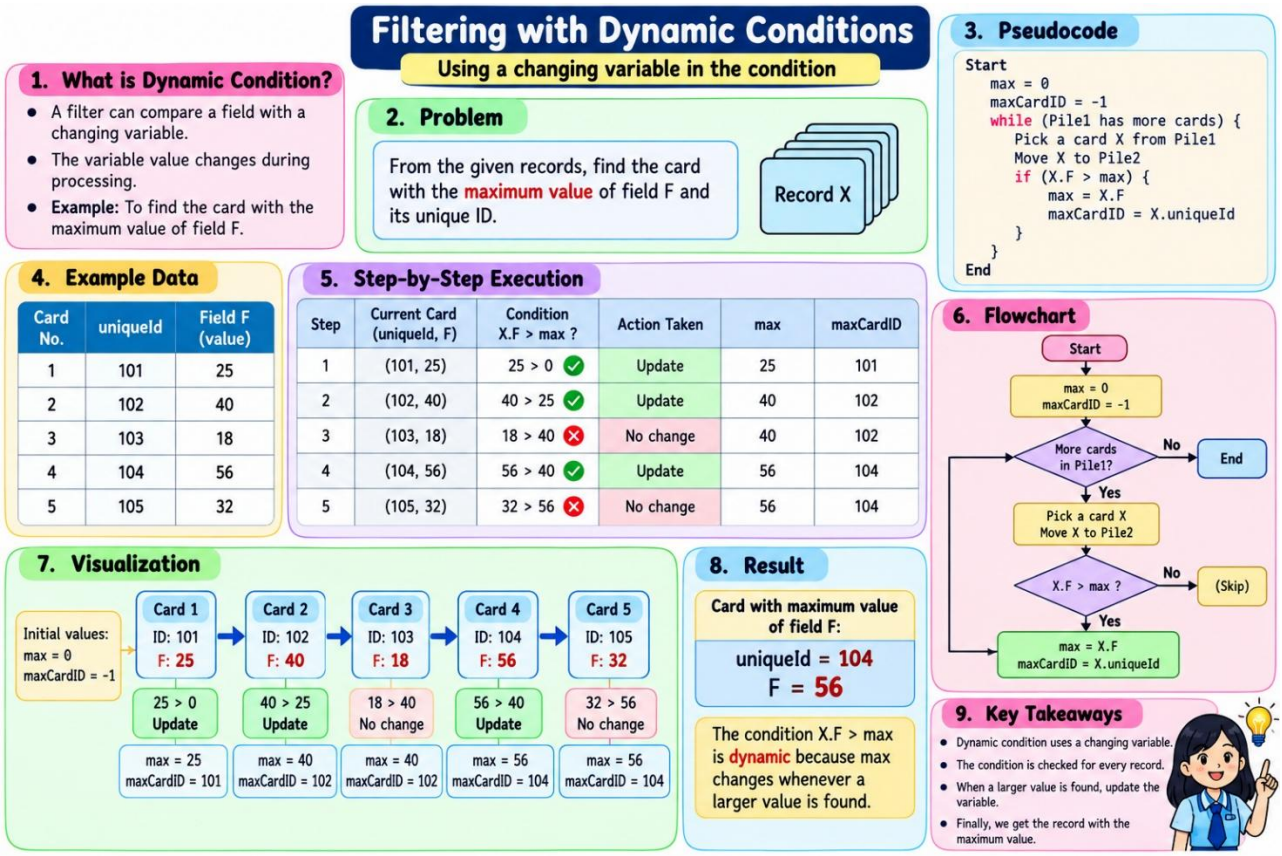
7. Filtering with dynamic conditions

A filter can also compare a field with a changing variable.

For example, to find the card with the maximum value of field F:

```
Start
    max = 0
    maxCardID = -1
    while (Pile1 has more cards) {
        Pick a card X from Pile1
        Move X to Pile2
        if (X.F > max) {
            max = X.F
            maxCardID = X.uniqueId
        }
    }
End
```

The condition X.F > max is dynamic because max changes whenever a larger value is found.



Quick revision

Filter type	Example condition	Meaning
Filter type	**Example condition**	**Meaning**
Simple	X.gender == "F"	Select girls
Compound with AND	X.gender == "F" AND X.townCity == "Chennai"	Select girls from Chennai
Compound with OR	X.mathsMarks > 90 OR X.physicsMarks > 90	Select students high in at least one subject
Dynamic	X.F > max	Select values larger than the current maximum

The main pattern is: **iterate through all records, test the filter condition, and update the result only for selected records.**